INTELLIGENT
CANDIDATE RANKING

Redrob Hackathon — AI Challenge

Beyond keyword matching — **understanding what the JD means**

Senior AI Engineer (Founding Team) · Redrob AI · 100,000 candidates

THE PROBLEM

Why keyword search fails this JD

KEYWORD STUFFERS

Marketing Manager listing 15 AI skill keywords. Top result of naive embedding search. Zero real fit.

PLAIN-LANGUAGE TIER 5s

Built a real recommendation system but profile says 'Backend Engineer'. Missed by skill matching.

GHOST CANDIDATES

Perfect-on-paper but inactive 200+ days, 5% recruiter response rate. Not actually available.

The dataset contains ~80 honeypot candidates and deliberately traps keyword-based systems.

ARCHITECTURE

Multi-signal composite ranker — no ML training, fully explainable, CPU in ~40 seconds

Title / Role	26%	Primary anti-keyword gate
Skills (7 buckets)	26%	With duration × endorsement trust multiplier
Experience	24%	YoE sweet-spot, product company %, prod-ML evidence
Availability	14%	last_active recency + response_rate + notice_period
Location	5%	India/Pune/Noida preferred, willing-to-relocate fallback
Education	3%	Tier × degree × field relevance
Engagement	2%	GitHub activity, saved by recruiters, verifications

HOW WE BEAT EACH TRAP

Keyword Stuffers

Title component (26%) gates the score. A Marketing Manager scores ≈0 regardless of AI skill keywords.

```
if "marketing manager" in title:
  return 0.0  # disqualified
```

Ghost Candidates

Availability sub-score (14%): last_active recency carries 30%. Inactive 180+ days → 0.2 recency multiplier.

```
days_inactive > 180:
  recency = 0.2  # severely downweighted
```

Honeypot Profiles

4-check detection: expert skills w/ 0 duration, YoE vs graduation year, future start dates, impossible career sums.

```
expert + duration_months==0 × 5+
  → HONEYPOT, removed from pool
```

SKILL TRUST MULTIPLIER

Claiming 'expert' with 0 months of usage should carry near-zero weight

```
trust = proficiency_base × duration_factor × endorsement_factor
                         × platform_assessment
```

Proficiency Base	
expert	1.00
advanced	0.80
intermediate	0.55
beginner	0.30

Duration Factor	
0 months (expert)	×0.35
< 6 months	×0.65
≥ 18 months	×1.05
≥ 36 months	×1.10

Endorsement Factor	
0 endorsements	×0.85
≥ 15 endorsements	×1.08
Platform score ≥80	×1.20
Platform score ≥60	×1.05

7 JD-required skill buckets: Embeddings · Vector DB · Ranking/IR · LLM/NLP · ML Frameworks · MLOps/Eval · Python

RESULTS

100K	7,585	40s	100
candidates processed	honeypots removed (7.6%)	runtime on CPU	top candidates ranked

TOP 5 CANDIDATES

#1	CAND_0018499	0.978	Senior ML Engineer · 7.2yr · Noida · 6/7 buckets · 15d notice · actively searching
#2	CAND_0002025	0.974	Senior AI Engineer · 5.9yr · Kerala · 7/7 buckets · 30d notice · short notice
#3	CAND_0081846	0.969	Lead AI Engineer · 6.7yr · Jaipur · 7/7 buckets · willing to relocate
#4	CAND_0008425	0.963	Senior NLP Engineer · 7.8yr · Kolkata · 7/7 buckets · all JD requirements met
#5	CAND_0077337	0.958	Staff ML Engineer · 7.0yr · Kochi · 6/7 buckets · open to work

DESIGN DECISIONS

Why rule-based over LLM-per-candidate

Why not call GPT-4 per candidate?

Compute constraint: 5min CPU, no network during ranking. 100K × LLM call = infeasible. Rule-based runs in 40 seconds.

Why not TF-IDF / BM25 between JD and profile?

The JD explicitly warns this is a trap. A Marketing Manager with 15 AI keywords scores top-10 on BM25. Title gating solves this.

Why not use vector embeddings?

Would embed and retrieve on the wrong signal (surface text vs. actual fit). Explainability lost. Runtime worse than rule-based for CPU.

Why bucket skills vs. counting keywords?

The JD has 7 distinct requirements. 3 Pinecone mentions counts as 1 bucket hit. Forces breadth-of-coverage, not keyword density.

WHAT WE BUILT

No LLMs at inference time	Pure Python stdlib, 40s on CPU, fully reproducible
7,585 honeypots caught	Conservative 4-check detector, 0 false-positive risk
Title-first scoring	The #1 anti-keyword-stuffer signal, per JD design intent
Behavioral availability layer	Dead accounts ranked down — only reachable candidates surface
Transparent reasoning	Every rank has specific, JD-connected, honest explanation

"The right answer involves reasoning about the gap between what the JD says and what the JD means."

— Job Description, Final Note for Hackathon Participants

```
reproduce: python rank.py --candidates ./candidates.jsonl --out ./submission.csv
```

Requirements: Python 3.8+, stdlib only · Runtime ≤ 40s · Memory < 2 GB